Esercizio 1 (11 punti).

Sia data una CPU con processore a **16GHz** e **8 CPI** (Clock per Instruction) che adoperi indirizzi da 32 bit e memoria strutturata su due livelli di **cache** (L1, L2), il cui setup è come segue:
L1 è una cache **set-associativa** a **2 vie** con **2 set** e **blocchi da 32 word**; adopera una politica di rimpiazzo **LRU**. Ricordiamo che consideriamo la linea come l'insieme del blocco in cache con tag e bit di validità, mentre il set è il gruppo di linee con il medesimo indice.
L2 è una cache **direct-mapped** con **4 linee** e **blocchi da 256 word**.

1) Supponendo che all'inizio nessuno dei dati sia in cache, indicare quali degli accessi in memoria indicati di seguito sono HIT o MISS in ciascuna delle due cache. Per ciascuna **MISS** indicare se sia di tipo Cold Start (**Cold**), Capacità (**Cap**) o Conflitto (**Conf**). Utilizzare la tabella sottostante per fornire i risultati e indicare la metodologia di calcolo.

	Address	1000	250	820	248	1500	600	1020	1380	8080	8000	1000	828
L1	Block#												
	Index												
	Tag												
	HIT/MISS												
	Miss type												
L2	Block#												
	Index												
	Tag												
	HIT/MISS												
	Miss type												

2) Calcolare le dimensioni in bit (compresi i bit di controllo ed assumendo che ne basti uno per la LRU, e ignorando il bit dirty) delle due cache: (a) L1 e (b) L2.
3) Assumendo che gli accessi in **memoria** impieghino **600 ns**, che gli **hit** nella cache **L1** impieghino **20 ns** e gli **hit** nella cache **L2** impieghino **40 ns**, calcolare (a) il **tempo totale** per la sequenza di accessi, (b) il tempo **medio** per la sequenza di accessi, e (c) **quante istruzioni** vengono svolte nel tempo medio calcolato.
4) Calcolare il word offset del sesto indirizzo per la cache L2 spiegando i calcoli effettuati.
5) Supponendo che gli indirizzi nella tabella siano virtuali e la memoria virtuale consti di **64 pagine** di **4KiB** ciascuna, indicarne i numeri di pagina virtuale.

Address	1000	250	820	248	1500	600	1020	1380	8080	8000	1000	828
Page#												

Esercizio 2 (11 punti).

Considerare l'architettura RISC-V a ciclo singolo nella figura in basso (e in allegato).
Si vuole aggiungere alla CPU l'istruzione **manysums** (`msum`), di **tipo R** e sintassi assembly:
`msum` **`a, b, c, d, m`**
Dove a, b, c, d, ed m sono rispettivamente i campi rs1, rs2, rd, funz7 e funz3 dell'istruzione.
In breve l'istruzione deve eseguire la seguente operazione: a0 ← (a+b+c+d) × 2^m. Più in dettaglio, l'istruzione deve operare come segue:
a) Intendendo a, b, c e d, come valori immediati (**ovvero ottenuti dall'istruzione direttamente e non dal banco dei registri**), essi vengono estesi a 32 bit **senza segno**. Invece, m può assumere valori solo da 0 a 3, ovvero vengono usati solo i due bit meno significativi di funz3 e intesi senza segno. Il bit più significativo di funz3 viene ignorato.
b) Viene scritto in a0 il valore risultante dalla somma dei primi quattro valori moltiplicati per 2^m.
c) Viene aggiornato il PC con il valore PC + 4.

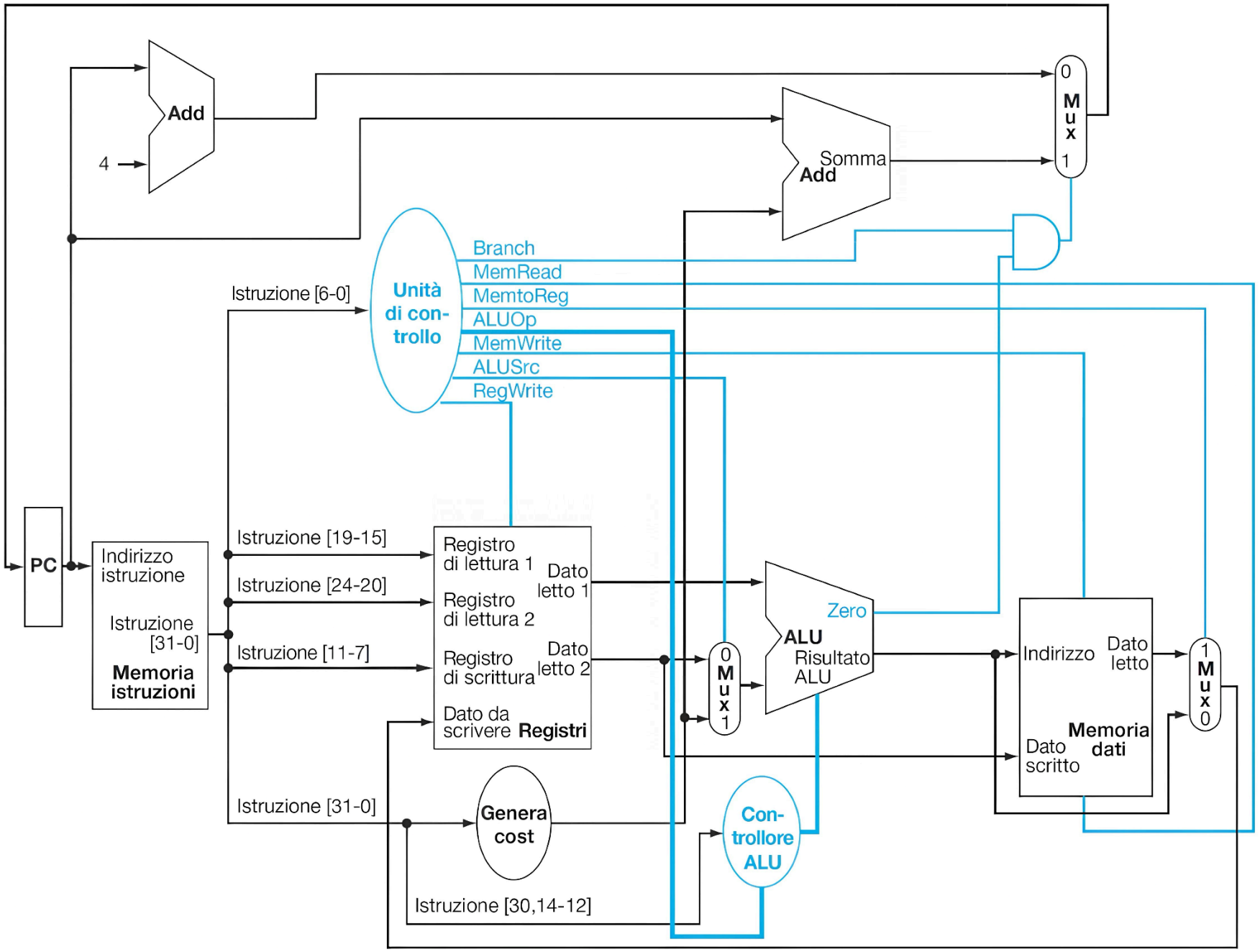
Esempio: Supponiamo che a=0, b=2, c=0b11111, d=30, e m = 3, ovvero venga eseguita l'istruzione `msum 0, 2, 31, 30, 3`
In tal caso:

- 2^3 × (0 + 2 + 31 + 30) = 504 verrà salvato in a0.
- Il PC verrà aggiornato con PC+4.

1) Mostrare le **modifiche all'architettura** della CPU RISC-V, avendo cura di aggiungere eventuali altri componenti necessari a realizzare l'istruzione. A tal fine, si può alterare la stampa del diagramma architetturale oppure ridisegnare i componenti interessati dalla modifica, avendo cura di indicare i fili di collegamento e tutti i segnali entranti ed uscenti. Indicare inoltre sul diagramma i **segnali di controllo** che la CU genera *per realizzare l'istruzione*.
   Si richiede di usare solo componenti elementari (MUX, shift logici di un valore costante, operatori logici e sommatori) per realizzare questa istruzione. Non può essere usato Genera Cost per estendere di segno i valori al punto a).
2) Indicare il contenuto in bit della word che esprime l'istruzione
   `msum` **`2, 3, 10, 0x7f, 2`**
   compilando la tabella sottostante (assumiamo che lo *OpCode* di `msum` sia `0x3A`, e che eventuali porzioni inutilizzate dall'istruzione siano codificate con zeri).



3) Supponendo che l'accesso alle **memorie** impieghi **200 ns**, l'accesso ai **registri 50 ns**, le operazioni dell'**ALU e** dei **sommatori 150 ns**, e che gli altri ritardi di propagazione dei segnali siano trascurabili, indicare la durata totale del **ciclo di clock** tale che anche l'esecuzione della nuova istruzione sia permessa *spiegando i calcoli effettuati*.
4) Descrivere nel dettaglio il funzionamento di Genera Cost, spiegando che operazioni svolge per ogni tipo di istruzione (R, I, S, SB, U, UJ) e come tali operazioni vengano implementate internamente.


Add
4
Somma
Add
0
M
u
x
1
Branch
MemRead
MemtoReg
ALUOp
MemWrite
ALUSrc
RegWrite
Unità di con-trollo
Istruzione [6-0]
PC
Indirizzo istruzione
Istruzione [31-0]
Memoria istruzioni
Istruzione [19-15]
Istruzione [24-20]
Istruzione [11-7]
Registro di lettura 1
Registro di lettura 2
Registro di scrittura
Dato da scrivere
Dato letto 1
Dato letto 2
Registri
Zero
ALU
Risultato ALU
Indirizzo
Dato letto
Dato scritto
Memoria dati
Istruzione [31-0]
Genera cost
Con-trollore ALU
Istruzione [30,14-12]

Esercizio 3 (11 punti)

Si consideri l'architettura RISC-V con pipeline. Il programma qui di seguito effettua la somma dei soli valori multipli di 4 degli elementi presenti nell'array vettore di lunghezza 10. Infine, viene stampato il valore di tale somma.

```
1   .data
2   vettore: .word 24, 1, 46, 54, 50, 12, 2, 11, 39, 4
3    # 7 non multipli di 4, 3 multipli di 4
4
5   .text                       # Sommo solo i multipli di 4
6   main:   addi t0, zero, 0    # somma parziale
7           auipc s0, 0         # operazione di tipo lui che fa s0 = PC + 0
8           addi s0, s0, 1020   # s0 ora contiene l'indirizzo base del vettore
9           addi t4, s0, 40     # primo indirizzo fuori dall'array
10  ciclo:  lw   t3, 0(s0)
11          andi t2, t3, 0x3    # è multiplo di 4?
12          bne  t2, zero, salta  # se non lo è, salto
13          add  t0, t0, t3     # altrimenti lo accumulo
14  salta:  addi s0, s0, 4      # prossimo elemento
15          blt  s0, t4, ciclo  # fine del ciclo?
16
17          addi a7,x0, 1       # stampa…
18          mv a0, t0           # … la somma
19          ecall
```

Si supponga che non si faccia uso di alcuna pseudoistruzione, e che le ecall non richiedano alcuno stallo. Si consideri che la decisione di branch venga presa in fase di ID, e che le unità di forwarding (quando usate) includano il forwarding da EXE a EXE, da MEM a EXE, e (per il branch) da EXE a ID.

Si indichino (ignorando hazard che possano concernere la ecall):

1) Per i primi 18 cicli di clock (**senza unità di forwarding**), le istruzioni tra le quali sono presenti **data hazard** – indicare i numeri di linea N ed M (visibili alla sinistra delle linee di codice in figura) ed il registro coinvolto xY (nel formato N / M / xY, ad esempio 17 / 18 / t0 se c'è un data hazard causato dal registro t0 tra l'istruzione alla linea 17 e quella alla linea 18);

2) Per i primi 18 cicli di clock (**con unità di forwarding**) le istruzioni tra le quali sono presenti **control hazard** – indicare i numeri di linea N ed M (nel formato N / M, ad esempio se l'istruzione alla linea 17 causa un control hazard con l'istruzione alla linea 23 scrivere: 17 / 23);

3) quanti **cicli di clock** sono necessari ad eseguire **con unità di forwarding**:

a) il **primo** ciclo spiegando il calcolo effettuato (indicando il tipo e numero di stalli)

b) **tutto** il programma spiegando il calcolo effettuato (usando la tabella fornita e indicando il tipo e numero di stalli)

sezione	cicli di clock richiesti	numero di occorrenze	totale
Totale			

4) quanti **cicli di clock** sarebbero necessari ad eseguire **senza forwarding**:

a) il **secondo** ciclo spiegando il calcolo effettuato (indicando il tipo e numero di stalli)
b) **tutto** il programma spiegando il calcolo effettuato (usando la tabella fornita e indicando il tipo e numero di stalli)

sezione	cicli di clock richiesti	numero di occorrenze	totale
Totale			

5) quali sono, per ognuna delle cinque fasi, le **istruzioni** (o le bolle) in pipeline durante il **15° ciclo di clock** (**con forwarding**);

IF:
ID:
EX:
MEM:
WB:

6) Descrivere i vantaggi e gli svantaggi dell'uso dell'architettura con pipeline rispetto all'architettura a singolo ciclo di clock.